

WikiContrib ER Design Document

Version 1.0 - Wikipedia evidence layer + analysis-system layer

Document field	Value
Project	WikiContrib - Wikipedia Contributor Impact
Document	Logical Entity-Relationship Design
Version	v1.0
Owner	Han Zhu (Han-ZhuAI)
Date	1 July 2026
Code baseline	main - merge commit bc08e4b
Status	Formal logical data-model baseline

Revision summary

- Separates the Wikipedia evidence layer from the WikiContrib analysis-system layer, following the two-model standard used in the Data Agent documentation.
- Documents every entity with What it is, Core fields and Why it is designed this way.
- Applies explicit 1:1, 1:N and N:M decomposition rules and preserves traceability from evidence to score and output artifact.
- This is a logical model for Python objects, cached inputs and JSON schemas; the current runtime does not require a relational database.

1. Two-layer model and decomposition rules

- Wikipedia evidence layer - public article revisions, Talk posts and deterministic evidence derived from them.
- WikiContrib analysis-system layer - evaluation scope, contributor profile, weight policy, rank, automation review, sensitivity result and output artifact.
- For 1:N relationships, the N-side is a separate entity. A 1:1 object is separated only when it has its own lifecycle or audit value.
- Every N:M relationship requires a junction or edge entity. TalkPost-to-TalkPost replies are represented by ReplyEdge.
- Derived metrics never overwrite source evidence. ContributorProfile and ImpactScore keep explicit upstream run, contributor and policy references.

MODEL BOUNDARY The diagrams are logical ER models. WikiContrib currently uses MediaWiki API responses, Python dataclasses and dictionaries, cached JSON and generated files - not PostgreSQL, MySQL or SQLite.

2. Wikipedia evidence-layer ER

This layer answers where the evidence comes from. Article, Revision, Contributor and Talk structures correspond to public MediaWiki data. RevisionDiff, TokenProvenance and WikitextDelta are deterministic evidence derived from that source history.

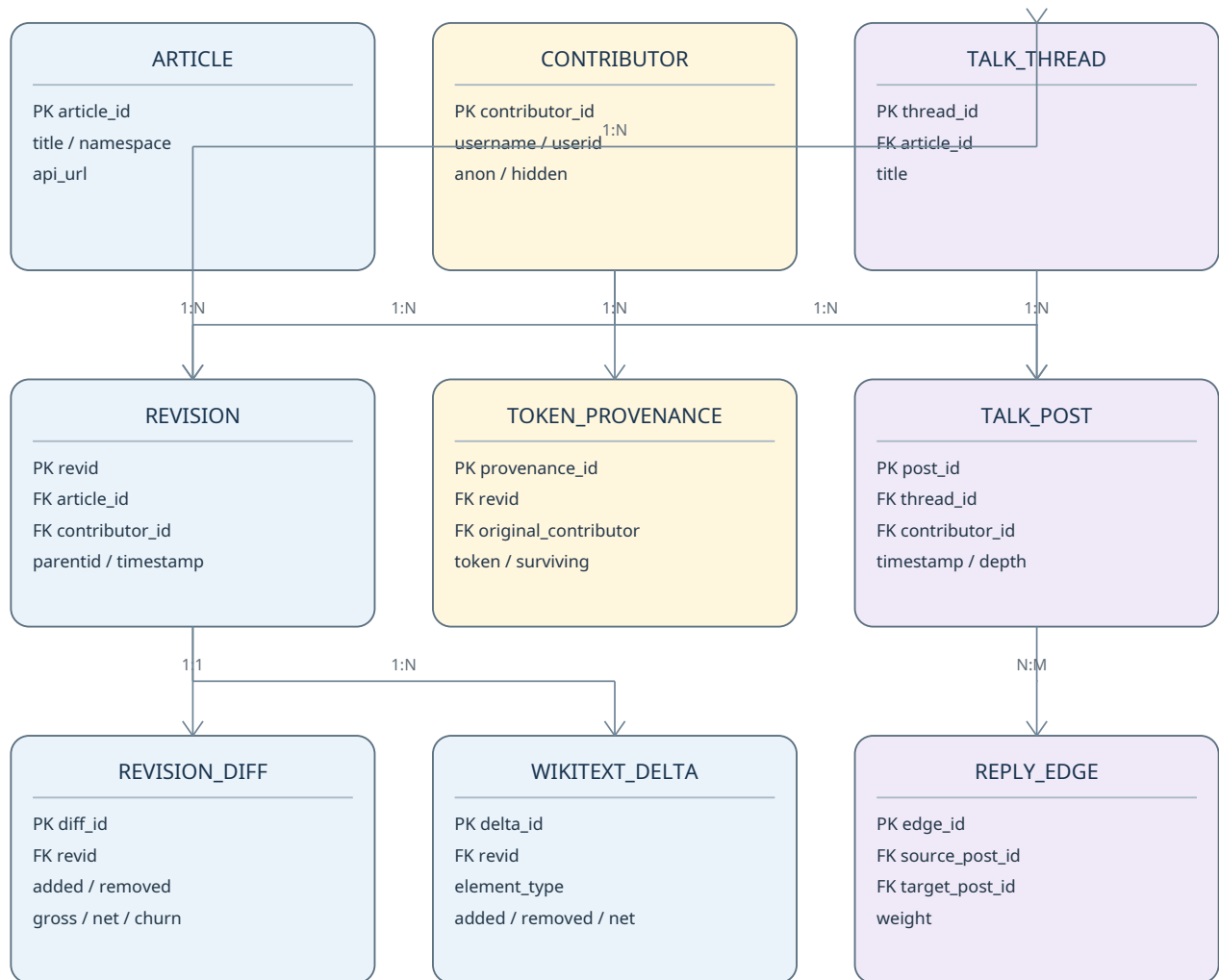


Figure 2-1. Wikipedia source data and derived evidence

2.1 Article, revision and identity

Article

What it is: The Wikipedia page that anchors one analysis.

Core fields: article_id, title, namespace, api_url

Why it is designed this way: Article is the common parent for revisions, Talk evidence and evaluation runs. It prevents page identity from being repeated inconsistently.

Revision

What it is: One complete wikitext state plus author and revision metadata.

Core fields: revid, article_id, contributor_id, parentid, timestamp, comment, minor, size

Why it is designed this way: Diff and provenance depend on chronological states. One Article has many Revisions, so Revision is an independent entity.

Contributor

What it is: A registered user, anonymous editor or hidden identity represented by one logical key.

Core fields: contributor_id, username, userid, identity_type, hidden

Why it is designed this way: Article and Talk evidence must aggregate on one identity key. Hidden identities remain placeholders rather than guessed real users.

2.2 Text-change and ownership evidence

RevisionDiff

What it is: The token-change and edit-classification evidence for one selected revision.

Core fields: diff_id, revid, added_tokens, removed_tokens, gross_words, net_words, churn, edit_type, revert_type

Why it is designed this way: A separate auditable record is required because volume and classification cannot be reconstructed from the final score alone.

TokenProvenance

What it is: Original contributor attribution, token position and final survival state for a word token.

Core fields: provenance_id, revid, original_contributor_id, token, position, surviving

Why it is designed this way: One revision can introduce or restore many tokens. Exact reverts and partial restorations must retain the original author instead of assigning ownership to the restorer.

WikitextDelta

What it is: Per-element added, removed and net counts for prose, heading, reference, template, table, category or link.

Core fields: delta_id, revid, contributor_id, element_type, added, removed, net

Why it is designed this way: Semantic evidence is separate from generic token diff so that every changed word receives one label without contaminating scoring inputs.

2.3 Talk evidence

TalkThread

What it is: A topic section within the article Talk page.

Core fields: thread_id, article_id, title, start_timestamp

Why it is designed this way: A Talk page contains multiple topics. Thread boundaries define reply context, so threads are not flattened into posts.

TalkPost

What it is: One signed and indented contribution parsed from a Talk thread.

Core fields: post_id, thread_id, contributor_id, timestamp, depth, text

Why it is designed this way: A thread has many posts and one contributor can write across threads. Timestamp and depth support reply inference and temporal analysis.

ReplyEdge

What it is: A weighted directed edge from the replying post/user to the replied-to post/user.

Core fields: edge_id, source_post_id, target_post_id, source_contributor_id, target_contributor_id, weight

Why it is designed this way: Replies form an N:M graph and therefore require an edge entity. Weight supports repeated replies and is the input to PageRank.

3. WikiContrib analysis-system ER

This layer answers how source evidence becomes a reproducible result. EvaluationRun is the center of the workflow, ContributorProfile stores normalized evidence, and ImpactScore joins a profile to an explicit WeightPolicy.

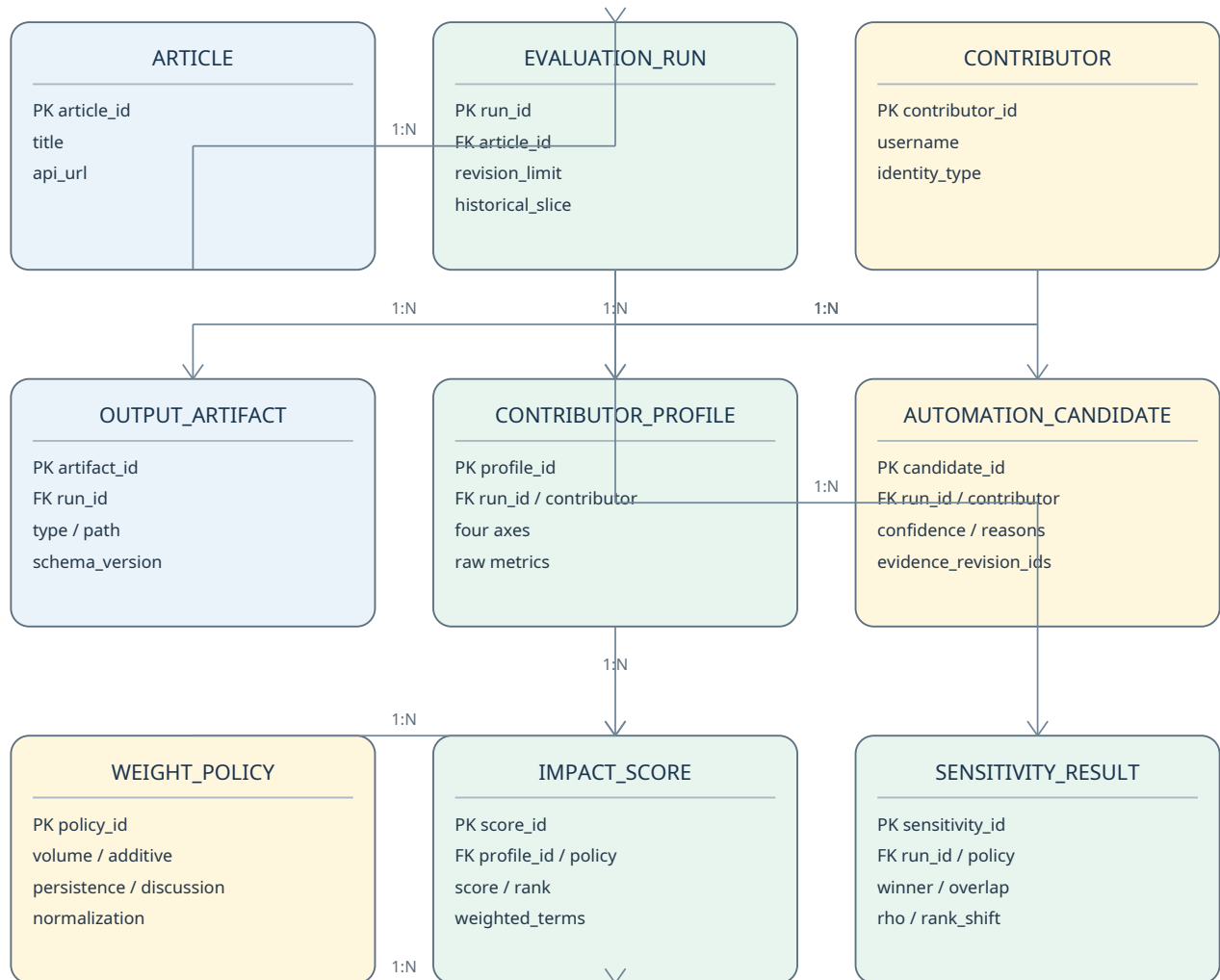


Figure 3-1. WikiContrib analysis, scoring and output model

3.1 Evaluation scope and profile

EvaluationRun

What it is: One complete analysis of one article under a defined revision scope.

Core fields: run_id, article_id, revision_limit, all_revisions, historical_slice, started_at, completed_at, schema_version

Why it is designed this way: Every result must distinguish earliest-N from full history. A run entity preserves scope, time, version and warnings across every downstream output.

ContributorProfile

What it is: Raw metrics and four normalized axes for one contributor in one run.

Core fields: profile_id, run_id, contributor_id, gross_words, additive_edits, surviving_words, pagerank, volume, additive, persistence, discussion

Why it is designed this way: Profiles are separated from scores so that the same evidence can be rescored under multiple policies without reprocessing revision history.

AutomationCandidate

What it is: An auditable indication that a contributor may be automated.

Core fields: candidate_id, run_id, contributor_id, confidence, reason_codes, evidence_revision_ids, excluded_from_ranking

Why it is designed this way: Automation detection is not an identity fact. Evidence and confidence remain visible, and exclusion affects ranking only.

3.2 Scoring policy and sensitivity

WeightPolicy

What it is: Four non-negative axis weights and their normalization rule.

Core fields: policy_id, name, volume_weight, additive_weight, persistence_weight, discussion_weight, normalized

Why it is designed this way: Weights are subjective policy, not profile evidence. A separate entity makes balanced and one-axis-emphasis policies reproducible.

ImpactScore

What it is: The score, rank and weighted terms for one profile under one policy.

Core fields: score_id, profile_id, policy_id, score, rank, volume_term, additive_term, persistence_term, discussion_term

Why it is designed this way: ContributorProfile and WeightPolicy are N:M. ImpactScore is the junction entity and preserves every term required to explain the total.

SensitivityResult

What it is: Cross-policy ranking-stability evidence for one run.

Core fields: sensitivity_id, run_id, policy_id, winner, top_k_overlap, spearman_rho, max_rank_shift

Why it is designed this way: Winner changes, overlap, rho and rank shift compare policies and do not belong to one contributor score.

3.3 Output and delivery

OutputArtifact

What it is: A JSON, PNG chart or Markdown evaluation file generated by one run.

Core fields: artifact_id, run_id, artifact_type, path, schema_version, created_at, checksum

Why it is designed this way: One run produces multiple files. Explicit artifacts retain run and schema traceability and support reproducible report generation.

4. Main relationships and cardinalities

Relationship	Cardinality	Design meaning
Article to Revision	1:N	One article has many chronological revisions.
Contributor to Revision	1:N	One contributor may author many revisions.
Revision to RevisionDiff	1:1	One selected revision produces one main diff summary.
Revision to TokenProvenance	1:N	One revision may introduce or restore many tokens.
Article to TalkThread to TalkPost	1:N:N	An article has threads and each thread has posts.
TalkPost to TalkPost	N:M	ReplyEdge expresses the directed reply graph.
Article to EvaluationRun	1:N	The same article may be evaluated under multiple scopes.
EvaluationRun to ContributorProfile	1:N	One run produces many contributor profiles.
ContributorProfile to WeightPolicy	N:M	ImpactScore joins profiles and policies.
EvaluationRun to OutputArtifact	1:N	One run may create JSON, charts and evaluation files.

5. How the two layers connect

- Scope resolution - EvaluationRun.article_id and revision_limit select Revision records; historical_slice propagates into every output.
- Text evidence - RevisionDiff, TokenProvenance and WikitextDelta aggregate by contributor into volume, additive and persistence profile inputs.
- Discussion evidence - TalkPost and ReplyEdge form a directed graph; PageRank aggregates into ContributorProfile.pagerank and normalizes to discussion.
- Scoring - ImpactScore multiplies the four ContributorProfile axes by normalized WeightPolicy values and stores each weighted term.
- Output traceability - OutputArtifact references EvaluationRun; JSON retains schema version, scope, weights, raw metrics and explanation.

6. Mapping to the current implementation

Logical entity	Current implementation	Persistence
Article / Revision	MediaWiki API response and cache	Cached JSON; no relational database
RevisionDiff / WikitextDelta	Python analysis-result objects	Aggregated into CLI and JSON
TokenProvenance	In-memory provenance-chain state	Final ownership and survival metrics
TalkThread / TalkPost / ReplyEdge	Talk parser and reply-graph objects	Edges and PageRank outputs
EvaluationRun	CLI arguments and output metadata	JSON metadata
ContributorProfile / ImpactScore	profile.py and scoring.py structures	CLI, JSON and charts
AutomationCandidate	Detection and ranking-layer filtering	Reasons and evidence in output
OutputArtifact	Local JSON, PNG and Markdown files	File system

7. Validation and limits

- Regression tests cover complete reverts and partial restoration so restored text is not incorrectly attributed to the restorer.
- Wikitext labels are mutually exclusive to prevent one changed word from being counted under multiple element types.
- ReplyEdge uses replier-to-replied-to direction; the Discussion axis uses PageRank with damping 0.85.
- ImpactScore retains weighted terms so every ranking can be recomputed from its profile and policy.
- The model does not solve cross-article absolute scoring, identity merging, Talk causality or relational-database migration.

VALIDATION BASELINE The main branch passes 245 tests with 95.45% total coverage. Python 3.10 and 3.12 CI jobs pass. Committed real-article evaluation covers Alan Turing, Kimchi and Python.